

Start the emulator from the command line

The Android SDK includes an Android device emulator—a virtual device that runs on your computer. The Android Emulator lets you develop and test Android apps without using a physical device.

This page describes command-line features that you can use with the Android Emulator. For information about using the Android Emulator UI, see [Run apps on the Android Emulator](/studio/run/emulator) (/studio/run/emulator).

Start the emulator

Use the `emulator` command to start the emulator, as an alternative to [running your project](/studio/run/emulator#runningapp) (/studio/run/emulator#runningapp) or [starting it through the AVD Manager](/studio/run/emulator#runningemulator) (/studio/run/emulator#runningemulator).

Here's the basic command-line syntax for starting a virtual device from a terminal prompt:

```
$ emulator -avd avd_name [ {-option [value]} ... ]
```

Or

```
$ emulator @avd_name [ {-option [value]} ... ]
```

For example, if you launch the emulator from within Android Studio running on a Mac, the default command line will be similar to the following:

```
tdelay none -netspeed full -qt-hide-window -grpc-use-token -idle-grpc-timeout
```

Please note that the arguments `-qt-hide-window -grpc-use-token -idle-grpc-timeout` are only used to run the emulator window within Android Studio. If you want to run the emulator on its own window, you should not use those extra parameters.

You can specify startup options when you start the emulator, but not after it has started.

For a list of AVD names, enter the following command:

```
$ emulator -list-avds
```

Use this option to display a list of AVD names from your Android home directory. You can override the default home directory by setting the `ANDROID_SDK_HOME` environment variable that specifies the root of the user-specific directory where all configuration and AVD content is stored.

You can set the environment variable in the terminal window before launching a virtual device or through your user settings in the operating system. For example, in your `.bashrc` file on Linux.

To stop the Android Emulator, close the emulator window.

Install an app

In addition to installing an app through Android Studio or the [emulator UI](#) (`/studio/run/emulator#tasks`), you can install your app on a virtual device by using the [adb](#) (`/tools/help/adb#move`) utility.

To use [adb](#) to install, run, and test your app, follow these general steps:

1. Build and package your app into an APK, as described in [Build and run your app](#) (`/studio/run`).
2. Start the emulator from the command line, as described in the previous section, using any startup options necessary.
3. Install your app using [adb](#) (`/tools/help/adb#move`).
4. Run and test your app on the emulator.

While the emulator is running, you can use the [Emulator console](#) (`/studio/run/emulator-console`) to issue commands as needed.

To uninstall an app, do so as you would on an Android device.

The virtual device preserves the app and its state data across restarts in a user data disk partition (`userdata-qemu.img`). To clear this data, start the emulator with the `-wipe-data` option or wipe the data in the AVD Manager. For more information about the user data partition and other storage, see the following section.

Note: The `adb` utility views the virtual device as an actual physical device. For this reason, you might have to use the `-d` flag with some common `adb` commands, such as `install`. The `-d` flag lets you specify which of several connected devices to use as the target of a command. If you don't specify `-d`, the emulator targets the first device in its list.

Understand the default directories and files

The emulator uses associated files, of which the AVD system and data directories are the most important. It's helpful to understand the emulator directory structure and files when specifying command-line options, although you typically don't need to modify the default directories or files.

The Android Emulator uses the ([QEMU](http://wiki.qemu.org/) (<http://wiki.qemu.org/>)) hypervisor.

AVD system directory

The system directory contains the Android system images that the emulator uses to simulate the operating system. This directory has platform-specific, read-only files shared by all AVDs of the same type, including API level, CPU architecture, and Android variant. The default locations are the following:

- macOS and Linux - `~/Library/Android/sdk/system-images/android-apiLevel/variant/arch/`
- Windows - `C:\Users\user\Library\Android\sdk\system-images\android-apiLevel\variant\arch\`

Where:

- `apiLevel` is a numeric API level or a letter for preview releases. For example, `android-V` indicates the Android VanillaIceCream preview. On release, it became API level 35, designated by `android-35`.

- `variant` is a name corresponding to specific features implemented by the system image. For example, `google_apis` or `android-wear`.
- `arch` is the target CPU architecture. For example, `x86`.

Use the `-sysdir` option to specify a different system directory for the AVD.

The emulator reads the following files from the system directory:

Table 1. System directory files read by the Android Emulator

File	Description	Option to specify a different file
<code>kernel-qemu</code> or <code>kernel-ranchu</code>	The binary kernel image for the AVD. <code>kernel-ranchu</code> is based on the QEMU emulator.	<code>-kernel</code>
<code>ramdisk.img</code>	The boot partition image. This is a subset of <code>system.img</code> loaded by the kernel initially before the system image is mounted. It typically contains just a few binaries and initialization scripts.	<code>-ramdisk</code>
<code>system.img</code>	The read-only, initial version of the system image. Specifically, the partition containing the system libraries and data corresponding to the API level and variant.	<code>-system</code>
<code>userdata.img</code>	The initial version of the data partition, which appears as <code>data/</code> in the emulated system and contains all writable data for the AVD. The emulator uses this file when you create a new AVD or use the <code>-wipe-data</code> option. For more information, see the <code>userdata-qemu.img</code> file description in the following section.	<code>-initdata</code> <code>-init-data</code>

AVD data directory

The AVD data directory, also called the content directory, is specific to a single AVD instance and contains all modifiable data for the AVD.

The default location is the following, where `name` is the AVD name:

- macOS and Linux - `~/ .android/avd/name.avd/`
- Windows 10, and higher - `C:\Users\user\.android\name.avd\`

Use the `-datadir` option to specify a different AVD data directory.

The following table lists the most important files contained in this directory:

Table 2. Important files in the AVD data directory

File	Description	Option to specify a different file
<code>userdata-qemu.img</code>	The content of the data partition, which appears as <code>data/</code> in the emulated system. When you create a new AVD or when you use the <code>-wipe-data</code> option to reset the AVD to the factory defaults, the emulator copies the <code>userdata.img</code> file in the system directory to create this file. Each virtual device instance uses a writable user-data image to store user and session-specific data. For example, it uses the image to store a unique user's installed app data, settings, databases, and files. Each user has a different <code>ANDROID_SDK_HOME</code> directory that stores the data directories for the AVDs created by that user. Each AVD has a single <code>userdata-qemu.img</code> file.	<code>-data</code>
<code>cache.img</code>	The cache partition image, which appears as <code>cache/</code> in the emulated system. It's empty when you first create an AVD or use the <code>-wipe-data</code> option. It stores temporary download files and is populated by the download manager and sometimes the system. For example, the browser uses it to cache downloaded web pages and images while the emulator is running. When you power off the virtual device, the file is deleted. You can persist the file by using the <code>-cache</code> option.	<code>-cache</code>
<code>sdcard.img</code>	(Optional) An SD card partition image that lets you simulate an SD card on a virtual device. You can create an SD card image file in the AVD Manager (<code>/studio/run/managing-avds</code>) or using the <code>mksdcard</code> (<code>/studio/command-line/mksdcard</code>) tool. The file is stored on your development computer and must be loaded at startup. When defining an AVD in the AVD Manager, you can choose to use an automatically managed SD card file or a file that you created with the <code>mksdcard</code> tool. You can view the <code>sdcard.img</code> file associated with an AVD in the AVD Manager. The <code>-sdcard</code> option overrides the SD card file specified in the AVD. Note that this SD Card option will not work on Mac with Apple Silicon. You can browse, send files to, and copy and remove files from a simulated SD card by using the emulator UI or the <code>adb</code>	<code>-sdcard</code>

(`/studio/command-line/adb#copyfiles`) utility while the virtual device is running. You can't remove a simulated SD card from a running virtual device.

To copy files to the SD card file before loading it, mount the image file as a loop device and then copy the files. Or, use a utility, such as the `mttools` package, to copy the files directly to the image.

The emulator treats the file as a pool of bytes, so the SD card format doesn't matter.

The `-wipe-data` option doesn't affect this file. If you want to clear the file, delete the file and then re-create it using the AVD Manager or the `mkcard` tool. Changing the size of the file also deletes the file and creates a new file.

List directories and files used by the emulator

You can discover where files are located in two ways:

- Use the `-verbose` or `-debug init` option when you start the emulator from the command line. Look at the output.
- Use the `emulator -help-option` command to list a default directory. For example:

```
$ emulator -help-datadir
```

```
Use '-datadir <dir>' to specify a directory where writable image files
will be searched. On this system, the default directory is:
```

```
/Users/me/.android
```

```
See '-help-disk-images' for more information about disk image files.
```

Command-line startup options

This section lists options you can supply on the command line when you start the emulator.

Note: The Android Emulator is continually under development to make it more reliable. For status on the issues reported against various command-line options and to report bugs, see the [Android Issue Tracker](https://issuetracker.google.com/issues?q=componentid:192708&s=modified_time:descfirst) (https://issuetracker.google.com/issues?q=componentid:192708&s=modified_time:descfirst).

Commonly used options

The following table lists command-line startup options that you might use more often:

Table 3. Commonly used command-line options

Command-line option	Description
Quick Boot	
<code>-no-snapshot-load</code>	Performs a cold boot and saves the emulator state on exit.
<code>-no-snapshot-save</code>	Performs a quick boot if possible, but does not save the emulator state on exit.
<code>-no-snapshot</code>	Disables the Quick Boot feature completely and doesn't load or save the emulator state.
Device Hardware	
<code>-camera-back mode</code> <code>-camera-front mode</code>	Sets the emulation mode for a camera facing back or front. This overrides any camera setting in the AVD. <code>mode</code> can be any of the following values: <code>emulated</code> - The emulator simulates a camera in the software. <code>webcamn</code> - The emulator uses a webcam connected to your development computer, specified by number. For a list of webcams, use the <code>-webcam-list</code> option. For example, <code>webcam0</code>. <code>none</code> - Disables the camera in the virtual device. For example: <pre>\$ emulator @Pixel8_API_34 -camera-back webcam0</pre>

-webcam-list

Lists the webcams on your development computer that are available for emulation. For example:

```
$ emulator @Pixel8_API_34 -webcam-list
List of web cameras connected to the computer:
Camera 'webcam0' is connected to device 'webcam0'
on channel 0 using pixel format 'UYVY'
```

In the example, the first **webcam0** is the name you use on the command line. The second **webcam0** is the name used by the OS on the development computer. The second name varies depending on the OS.

As of SDK Tools 25.2.4, the AVD name is required.

Disk images and memory

-memory size

Specifies the physical RAM size, from 1536 to 8192 MBs. For example:

```
$ emulator @Pixel8_API_34 -memory 2048
```

This value overrides the AVD setting.

-sdcard *filepath*

Specifies the filename and path to an SD card partition image file. For example:

```
$ emulator @Pixel8_API_34 -sdcard C:/sd/sdcard.img
```

If the file isn't found, the emulator still launches, but without an SD card. The command returns a **No SD Card Image** warning.

If you don't specify this option, the default is **sdcard.img** in the data directory unless the AVD specifies something different. For details about emulated SD cards, see [AVD data directory](#) (`#data-filedir`).

-wipe-data

Deletes user data and copies data from the initial data file. This option clears the data for the virtual device and returns it to the same state as when it was first defined. All installed apps and settings are removed. For example:

```
$ emulator @Pixel8_API_34 -wipe-data
```

By default, the user data file is `userdata-qemu.img` and the initial data file is `userdata.img`. Both of these files reside in the data directory. The `wipe-data` option doesn't affect the `sdcard.img` file. For more information about user data, see the section called [Understand the default directories and files](#) (#filedir).

Debug**-debug *tags***

Enables or disables the display of debug messages for one or more tags. Separate multiple tags by a space, comma, or column. For example:

```
$ emulator @Pixel8_API_34 -debug init,metrics
```

To disable a tag, place a dash (-) in front of it. For example, the following option displays all debug messages, except the ones related to network sockets and metrics:

```
-debug all,-socket,-metrics
```

For a list of tags and descriptions, use the `-help-debug-tags` option. For example:

```
$ emulator -help-debug-tags
```

You can define the default debug tags in the `ANDROID_VERBOSE` (/studio/command-line/variables#android_verbose) environment variable. Define the tags you want to use in a comma-delimited list. Here's an example that shows how to specify the `socket` and `gles` tags:

```
ANDROID_VERBOSE=socket,gles
```

It's equivalent to using:

```
-debug-socket -debug-gles
```

or

```
-debug socket,gles
```

```
-debug-tag  
-debug-no-tag
```

Enables a specific debug message type. Use the `no` form to disable a debug message type. For example:

```
$ emulator @Pixel8_API_34 -debug-all -debug-no-metrics
```

For a list of tags, use the `emulator -help-debug-tags` command.

```
-logcat logtags
```

Enables the display of Logcat messages for one or more tags and writes them to the terminal window. For example, the following command enables error messages from all components:

```
$ emulator @Pixel8_API_34 -logcat *:e
```

`logtags` uses the same format as the `adb logcat logtags` command. Enter `adb logcat -help` for more information. It's a list of space- or comma-separated log filters of the format `componentName:logLevel`. `componentName` is either a wildcard asterisk (*) or a component name, such as `ActivityManager`, `SystemServer`, `InputManager`, or `WindowManager`.

`logLevel` is one of these values:

- `v` - verbose
- `d` - debug
- `i` - informative
- `w` - warning log level

- **e** - error
- **s** - silent

The following example displays GSM component messages at the informative log level:

```
$ emulator @Pixel8_API_34 -logcat '*:s GSM:i'
```

If you don't supply the **-logcat** option on the command line, the emulator looks for the **ANDROID_LOG_TAGS** (/studio/command-line/variables#android_log_tags) environment variable. If **ANDROID_LOG_TAGS** is defined with a valid **logtags** value and isn't empty, the emulator uses its value to enable Logcat output to the terminal by default. You can also redirect the same or other log messages to the terminal through **adb**.

For more information about Logcat and **adb**, see [Logcat command-line tool](#) (/studio/command-line/logcat), [View and write logs with Logcat](#) (/studio/debug/am-logcat), [Log](#) (/reference/android/util/Log) class, and [Issue adb commands](#) (/studio/command-line/adb#issuingcommands).

-show-kernel

Displays kernel debug messages in the terminal window. For example:

```
$ emulator @Pixel8_API_34 -show-kernel
```

One use of this option is to check that the boot process works correctly.

-verbose

Prints emulator initialization messages to the terminal window. For example

```
$ emulator @Pixel8_API_34 -verbose
```

It displays which files and settings are actually selected when starting a virtual device defined in an AVD. This option is the same as specifying **-debug-init**.

Network

-dns-server *servers* Uses the specified DNS servers. ***servers*** is a comma-separated list of up to four DNS server names or IP addresses. For example:

```
$ emulator @Pixel8_API_34 -dns-server 192.0.2.0,
192.0.2.255
```

By default, the emulator tries to detect the DNS servers you're using and sets up special aliases in the emulated firewall network to allow the Android system to connect directly to the servers. Use the **-dns-server** option to specify a different list of DNS servers.

-http-proxy *proxy* Makes all TCP connections through a specified HTTP/HTTPS proxy. If your emulator must access the internet through a proxy server, you can use this option or the **http_proxy** environment variable to set up the appropriate redirection. For example:

```
$ emulator @Pixel8_API_34 -http-proxy myserver:1981
```

proxy can be one of the following:

http://*server:port*

http://*username:password@server:port*

The **http://** prefix can be omitted.

If this option isn't supplied, the emulator looks up the **http_proxy** environment variable and automatically uses any value matching the ***proxy*** format. For more information, see [Using the emulator with a proxy](#). (/studio/run/emulator-networking#proxy).

-netdelay *delay* Sets network latency emulation to one of the following ***delay*** values in milliseconds:

- **gsm** - GSM/CSD (min 150, max 550).
- **hscsd** - HSCSD (min 80, max 400).
- **gprs** - GPRS (min 35, max 200).

- **edge** - EDGE/EGPRS (min 80, max 400).
- **umts** - UMTS/3G (min 35, max 200).
- **hsdpa** - HSDPA (min 0, max 0).
- **lte** - LTE (min 0, max 0).
- **evdo** - EVDO (min 0, max 0).
- **none** - No latency, the default (min 0, max 0).
- **num** - Specifies exact latency.
- **min:max** - Specifies individual minimum and maximum latencies.

For example:

```
$ emulator @Pixel8_API_34 -netdelay gsm
```

The emulator supports network throttling as well as higher connection latencies. You can define it either through the skin configuration or with the **-netspeed** and **-netdelay** options.

-netfast

Disables network throttling. For example:

```
$ emulator @Pixel8_API_34 -netfast
```

This option is the same as specifying **-netspeed full -netdelay none**. These are the default values for these options.

-netspeed speed

Sets the network speed emulation. Specifies the maximum network upload and download speeds with one of the following **speed** values in kbps:

- **gsm** - GSM/CSD (up: 14.4, down: 14.4).
- **hscsd** - HSCSD (up: 14.4, down: 57.6).
- **gprs** - GPRS (up: 28.8, down: 57.6).
- **edge** - EDGE/EGPRS (up: 473.6, down: 473.6).
- **umts** - UMTS/3G (up: 384.0, down: 384.0).
- **hsdpa** - HSDPA (up: 5760.0, down: 13,980.0).

- `lte` - LTE (up: 58,000, down: 173,000).
- `evdo` - EVDO (up: 75,000, down: 280,000).
- `full` - No limit, the default (up: 0.0, down: 0.0).
- `num` - Specifies both upload and download speed.
- `up:down` - Specifies individual up and down speeds.

For example:

```
$ emulator @Pixel8_API_34 -netspeed edge
```

The emulator supports network throttling as well as higher connection latencies. You can define it either through the skin configuration or with the `-netspeed` and `-netdelay` options.

`-port port`

Sets the TCP port number that's used for the console and `adb`. For example:

```
$ emulator @Pixel8_API_34 -port 5556
```

The default value is 5554 for the first virtual device instance running on the your machine. A virtual device normally occupies a pair of adjacent ports: a console port and an `adb` port. The console of the first virtual device running on a particular machine uses console port 5554 and `adb` port 5555. Subsequent instances use port numbers increasing by two. For example, 5556/5557, 5558/5559, and so on. The range is 5554 to 5682, allowing for 64 concurrent virtual devices.

The port assignments are often the same as specifying `-ports port, {port + 1}`. `{port + 1}` must be free and is reserved for `adb`. If any of the console or `adb` ports are already in use, the emulator won't start.

The `-port` option reports which ports and serial number the virtual device is using and warns if there are any issues with the values you provided. In the emulator UI, you can see the console port number in the window title and view the `adb` port number by selecting **Help > About**.

Note that if the `port` value is not even and is in the range 5554 to 5584, the virtual device will start but not be visible when you use the `adb devices`

command, if the `adb server` starts after the emulator. For this reason, we recommend using an even console port number.

`-ports`
`console-port, adb-`
`port`

Sets the TCP ports used for the console and `adb`. For example:

```
$ emulator @Pixel8_API_34 -ports 5556,5559
```

The valid ports range is 5554 to 5682, allowing for 64 concurrent virtual devices. The `-ports` option reports which ports and serial number the emulator instance is using and warns if there are any issues with the values you provided.

We recommend using the `-port` option instead, where possible. The `-ports` option is available for network configurations that require special settings.

For more information about setting console and `adb` ports, see the `-port` option.

`-tcpdump` *filepath*

Captures network packets and stores them in a file. For example:

```
$ emulator @Pixel8_API_34 -tcpdump /path/dumpfile.cap
```

Use this option to begin capturing all network packets that are sent through the virtual Ethernet LAN of the emulator. Afterward, you can use a tool like Wireshark to analyze the traffic.

Note that this option captures all ethernet packets and isn't limited to TCP connections.

System

`-accel` *mode*

Configures emulator VM acceleration. For example:

```
$ emulator @Pixel8_API_34 -accel auto
```

Accelerated emulation works for x86 and x86_64 system images only. On Linux, it relies on KVM. On Windows and Mac, it relies on an Intel CPU and Intel HAXM driver. This option is ignored if you're not emulating an x86 or x86_64 device.

Valid values for `mode` are:

- `auto` - Determines automatically if acceleration is supported and uses it when possible (default).
- `off` - Disables acceleration entirely, which is primarily useful for debugging.
- `on` - Forces acceleration. If KVM or HAXM isn't installed or usable, the emulator won't start and prints an error message.

For more information, see [Configure hardware acceleration for the Android Emulator \(/studio/run/emulator-acceleration\)](/studio/run/emulator-acceleration).

`-accel-check`

Checks whether a required hypervisor for emulator VM acceleration is installed (HAXM or KVM). For example:

```
$ emulator -accel-check
```

For more information, see [Check if a hypervisor is installed \(/studio/run/emulator-acceleration#accel-check\)](/studio/run/emulator-acceleration#accel-check).

`-engine engine`

Specifies the emulator engine:

- `auto` - Automatically selects an engine (default).
- `classic` - Uses the older QEMU 1 engine (deprecated).
- `qemu2` - Uses the newer QEMU 2 engine.

For example:

```
$ emulator @Pixel8_API_34 -engine auto
```

Auto-detection should choose the value that provides the best performance when emulating a particular AVD. Use the `-engine` option for debugging and comparison purposes only.

-gpu *mode*

Selects the GPU emulation mode. For example:

```
$ emulator @Pixel8_API_34 -gpu swiftshader_indirect
```

For more information, see [Configure graphics acceleration](#) (/studio/run/emulator-acceleration#accel-graphics).

-no-accel

Disables emulator VM acceleration when using an x86 or x86_64 system image. It's useful for debugging only and is the same as specifying **-accel off**. For example:

```
$ emulator @Pixel8_API_34 -no-accel
```

For more information, see [Configure hardware acceleration for the Android Emulator](#) (/studio/run/emulator-acceleration).

**-nojni
-no-jni**

Disables extended Java Native Interface (JNI) checks in the Android Dalvik or ART runtime. For example:

```
$ emulator @Pixel8_API_34 -nojni
```

When you start a virtual device, extended JNI checks are enabled by default. For more information, see [JNI tips](#) (/training/articles/perf-jni).

**-selinux
{disabled|permissive}**

Sets the Security-Enhanced Linux ([SELinux](#) (https://en.wikipedia.org/wiki/Security-Enhanced_Linux)) security module to either **disabled** or **permissive** mode on a Linux operating system. For example:

```
me-linux$ emulator @Pixel8_API_34 -selinux permissive
```

By default, SELinux is in **enforcing** mode, meaning the security policy is enforced. **permissive** mode loads the SELinux policy but doesn't enforce it. This option only logs policy violations. **disabled** mode disables kernel support for SELinux.

-timezone *timezone*

Sets the time zone for the virtual device to ***timezone*** instead of the host time zone. For example:

```
$ emulator @Pixel8_API_34 -timezone Europe/Paris
```

By default, the emulator uses the time zone of your development computer. Use this option to specify a different time zone or if the automatic detection isn't working correctly. The ***timezone*** value must be in zoneinfo (https://en.wikipedia.org/wiki/List_of_tz_database_time_zones) format, which is ***area/location*** or ***area/subarea/location***. For example:

- **America/Los_Angeles**
- **Europe/Paris**
- **America/Argentina/Buenos_Aires**

The specified time zone must be in the zoneinfo database (<https://www.iana.org/time-zones>).

-version

Displays the emulator version number. For example:

```
$ emulator @Pixel8_API_34 -version
```

Or

```
$ emulator -version
```

UI

-no-boot-anim

Disables the boot animation during emulator startup for faster booting. For example:

```
$ emulator @Pixel8_API_34 -no-boot-anim
```

On slower computers, this option can significantly speed up the boot sequence.

-screen mode

Sets emulated touch screen mode. For example:

```
$ emulator @Pixel8_API_34 -screen no-touch
```

mode can be any of the following values:

- **touch** - Emulates a touch screen (default).
- **multi-touch** - Emulates a multi-touch screen.
- **no-touch** - Disables touch and multi-touch screen emulation.

Advanced options

The command-line startup options in the following table are available but not commonly used by the average app developer.

In the descriptions, the *working directory* is the current directory in the terminal where you're entering commands. For information about the AVD system directory and data directory and the files stored within them, see the section about [default directories and files](#) (#filedir).

Some of these options are appropriate for external app developers, and some of them are used primarily by platform developers. *App developers* create Android apps and run them on specific AVDs. *Platform developers* work on the Android system and run it inside the emulator with no pre-created AVD.

Table 4. Advanced command-line options

Advanced option	Brief description
-----------------	-------------------

-bootchart
timeout Enables bootcharting with a timeout in seconds. Some Android system images have a bootcharting facility. You can pass a bootcharting timeout period to the system. If your init system doesn't have bootcharting activated, the option does nothing. This option is for platform developers, not external app developers.

For example:

```
$ emulator @Pixel8_API_34 -bootchart 120
```

-cache
filepath Specifies a cache partition image file. Provides a filename and an absolute path or a path relative to the AVD directory to set up a persistent cache file. If the file doesn't exist, the emulator creates it.

For example:

```
$ emulator @Pixel8_API_34 -cache  
  ~/.android/avd/Pixel8_API_34.avd/cache_persistent.img
```

If you don't use this option, the default is a temporary file named **cache.img**. For more information, see [AVD data directory](#) (`#data-filedir`).

-cache-size
size Sets the cache partition size in MBs.

For example:

```
$ emulator @Pixel8_API_34 -cache-size 1000
```

If you don't specify this option, the default is 66 MB. Normally, most app developers don't need to download very large files that are larger than the default cache. For more information, see [AVD data directory](#) (`#data-filedir`).

-data
filepath Sets the user data partition image file. Provides a filename and an absolute path or a path relative to the AVD directory to set up a persistent user data file. If the file doesn't exist, the emulator creates a default **userdata.img** file, stores it in the filename you specified, and persists user data.

For example:

```
$ emulator @Pixel8_API_34 -data  
~/.android/avd/Pixel8_API_34.avd/userdata-test.img
```

If you don't use this option, the default is a file named `userdata-qemu.img`. For more information about the user data file, see [AVD data directory](#) (`#data-filedir`).

`-datadir`

Specifies a data directory using an absolute path. For more information, see [AVD data directory](#).

For example:

```
$ emulator @Pixel8_API_34 -datadir  
~/.android/avd/Pixel8_API_34.avd/mytest
```

`-force-32bit`

Uses the 32-bit emulator on 64-bit platforms. Occasionally, this option is useful for testing. For example, there was an issue where the emulator would sometimes not run on 64-bit Windows. This option was helpful for performing comparisons to debug the issue. Here's an example:

```
$ emulator @Pixel8_API_34 -force-32bit
```

`-help-disk-images` Gets help about disk images. This option provides information relevant to both application developers and system developers. For example:

```
$ emulator -help-disk-images
```

`-help-char-devices` Gets help about character `device` specifications. A `device` parameter is required by some devices. For example:

```
$ emulator -help-char-devices
```

-help-sdk-images Gets help about disk images relevant to app developers. This option gets information about images located for an AVD created with the SDK tools. For example:

```
$ emulator -help-sdk-images
```

-help-build-images Gets help about disk images relevant to platform developers. For example:

```
$ emulator -help-build-images
```

-initdata filepath Specifies the initial version of the data partition. After wiping user data, the emulator copies the specified file to user data (by default, the **userdata-qemu.img** file) instead of using the **userdata.img** file as the initial version. Specifies the filename and an absolute path or working directory.

For example:

```
$ emulator @Pixel8_API_34 -initdata
~/Library/Android/sdk/system-images/android-34/
google_apis/x86_64/userdata-test.img
```

If you don't specify a path, it places the file in the system directory. For more information, see [directory](#) (`#system-filedir`).

-kernel filepath Uses a specific emulated kernel. If you don't specify a path, the emulator looks in the system directory. Use the **-show-kernel** option to view kernel debug messages.

For example:

```
$ emulator @Pixel8_API_34 -kernel
~/Library/Android/sdk/system-images/android-34/
google_apis/x86_64/kernel-test.img -show-kernel
```

If you don't specify this option, the default is `kernel-ranchu`. For more information, see [\(#system-filedir\)](#).

`-noaudio`
`-no-audio`

Disables audio support for this virtual device. Some Linux and Windows computers have cause different symptoms, such as preventing the emulator from starting. In this case, use `-no-audio` to overcome the issue. Alternatively, you can use the `QEMU_AUDIO_DRV` environment variable to specify a different audio backend.

For example:

```
$ emulator @Pixel8_API_34 -noaudio
```

`-nocache`
`-no-cache`

Starts the emulator without a cache partition. If you don't use this option, the default is `cache.img`. This option is for platform developers only. For more information, see [AVD configuration](#) ([#data-filedir](#)).

For example:

```
$ emulator @Pixel8_API_34 -nocache
```

`-no-snapshot`

Inhibits both the automatic load and save operations, causing the emulator to execute a full boot to lose its state when closed. It overrides the `-snapshot` option.

For example:

```
$ emulator @Pixel8_API_34 -no-snapshot
```

`-no-snapshot-load`

Prevents the emulator from loading the AVD state from snapshot storage. Performs a full boot.

For example:

```
$ emulator @Pixel8_API_34 -no-snapshot-load
```

-no-snapshot-save

Prevents the emulator from saving the AVD state to snapshot storage on exit, meaning the state is lost.

For example:

```
$ emulator @Pixel8_API_34 -no-snapshot-save
```

-no-snapshot-update-time

Doesn't try to correct the AVD clock time immediately on snapshot restore. This option is useful for testing as it avoids a sudden time jump. Time updates are still sent to the AVD about every second.

For example:

```
$ emulator @Pixel8_API_34 -no-snapshot-update-time
```

-no-snapstorage

Starts the emulator without mounting a file to store or load state snapshots, forcing a full snapshot functionality. This option overrides the **-snapstorage** and **-snapshot** options.

For example:

```
$ emulator @Pixel8_API_34 -no-snapstorage
```

-no-window

Disables graphical window display on the emulator. This option is useful when running tests that have no display. You can access the emulator through **adb** or the console. For example:

```
$ emulator @Pixel8_API_34 -no-window
```

-partition-size size

Specifies the system data partition size in MBs. For example:

```
$ emulator @Pixel8_API_34 -partition-size 1024
```

-prop **name=value** Sets an Android system property in the emulator when it boots. **name** must be a property **qemu_prop** of at most 32 characters, without any spaces, and **value** must be a string (For an example, see the [property_contexts file](https://android.googlesource.com/device/generic/goldfish/+refs/heads/main/sepolicy/) (<https://android.googlesource.com/device/generic/goldfish/+refs/heads/main/sepolicy/>). You can specify several **-prop** options on one command line. This option can be useful in the following example:

```
$ emulator @Pixel8_API_34 -prop qemu.name=value -prop qemu.abc=xyz
```

-qemu args Passes arguments to the QEMU emulator software. When using this option, make sure it's specified, as all options after it are interpreted as QEMU-specific options. This option is intended to be used only by developers who are very familiar with QEMU and Android emulation.

-qemu -h Displays **-qemu** help. For example:

```
$ emulator -qemu -h
```

-ramdisk filepath Specifies a ramdisk boot image. Specifies the filename and an absolute path or a path relative to the system directory.

For example:

```
$ emulator @Pixel8_API_34 -ramdisk
~/Library/Android/sdk/system-images/android-34/
google_apis/x86_64/ramdisk-test.img
```

If you don't use this option, the default is the **ramdisk.img** file in the system directory. For more information, see [AVD system directory](#) (`#system-filedir`).

-report-console socket Reports the console port to a remote third party before starting emulation. It can be used in a testing script. **socket** must use one of these formats:

- `tcp:port[, server][, max=seconds][, ipv6]`
- `unix:port[, server][, max=seconds][, ipv6]`

For more information, use the `-help-report-console` option as described in the [specific options](#) (#help-detailed).

-shell

Creates a root shell console on the current terminal. This option differs from the `adb shell` (/studio/command-line/adb#shellcommands) command in the following ways:

- It creates a *root* shell that lets you modify many parts of the system.
- It works even if the `adb daemon` in the emulated system is broken.
- Press Control+C (or Command-C, on macOS) to stop the emulator instead of the shell.

For example:

```
$ emulator @Pixel8_API_34 -shell
```

**-snapshot
name**

Specifies the name of a snapshot within a snapshot storage file for automatic start and save.

Rather than executing a full boot sequence, the emulator can resume execution from an existing snapshot, which is usually significantly faster. When you supply this option, the emulator loads the snapshot from the snapshot image and saves it back under the same name on exit.

If you don't use this option, the default is a full boot sequence. If the specified snapshot exists, the emulator performs a full boot sequence instead and performs a save operation.

See the `-snapstorage` option for information on specifying a snapshot storage file and path.

```
$ emulator @Pixel8_API_34 -snapshot snapshot2
```

Remember that in the process of loading a snapshot, all contents of the system, user data, and apps are overwritten with the contents they held when the snapshot was made. Unless you save a different snapshot, any changes since then are lost.

You can also create a snapshot from the emulator console by using the `avd snapshot` command. For more information, see [Send Emulator console commands](#) (/studio/run/emulator-console).

**-snapshot-
list**

Displays a list of available snapshots. This command prints a table of snapshots that are stored in the snapshot storage file that the emulator was started with, then exits. If you specify `-snapstorage`, the command prints a table of the snapshots stored in file.

For example:

```
$ emulator @Pixel8_API_34 -snapshot-list -snapstorage
~/.android/avd/Pixel8_API_34.avd/snapshots-test.img
```

You can use the ID and TAG column values in the output as arguments for the `-snapshot`

- Specifies a repository file that contains all state snapshots. All snapshots made during the emulator run are saved to this file. Only snapshots in this file can be restored during the emulator run.

filepath

For example:

```
$ emulator @Pixel8_API_34 -snapstorage
~/.android/avd/Pixel8_API_34.avd/snapshots-test.img
```

If you don't specify this option, the default is `snapshots.img` in the data directory. If the file doesn't exist, the emulator will start, but without support for saving or loading state snapshots.

-sysdir Specifies a system directory using an absolute path. For more information, see [AVD system directory](#) (`#system-filedir`). For example:

```
$ emulator @Pixel8_API_34 -sysdir
~/Library/Android/sdk/system-images/android-34/
google_apis/x86_64/test
```

-system Specifies an initial system file. Provides the filename and an absolute path or a path relative to the system directory.

filepath

For example:

```
$ emulator @Pixel8_API_34 -system
~/Library/Android/sdk/system-images/android-34/
google_apis/x86_64/system-test.img
```

If you don't use this option, the default is the `system.img` file in the system directory. For AVD system directory (`#system-filedir`).

-use-system-libs On Linux, uses the system `libstdc++` instead of the version bundled with the emulator only if the emulator won't start normally, and note that it doesn't always work. Alternative `ANDROID_EMULATOR_USE_SYSTEM_LIBS` (/studio/command-line/variables#android_en environment variable to 1.

For example:

```
me-linux$ emulator @Pixel8_API_34 -use-system-libs
```

-writable-system Use this option to have a writable system image during your emulation session. To do so

1. Start a virtual device with the `-writable-system` option.
2. Enter the `adb remount` command from a command terminal to tell the emulator to r read/write. By default, it's mounted as read-only.

Using this flag creates a temporary copy of the system image that can be very large, up but will be destroyed when the emulator exits.

Deprecated options

The following command-line options are deprecated:

- `-audio-in`
- `-audio-out`
- `-charmap`
- `-code-profile`
- `-cpu-delay`
- `-dpi-device`
- `-dynamic_skin`
- `-enable-kvm`
- `-gps`
- `-image`

- `-keyset`
- `-help-keys`
- `-help-keyset-file`
- `-nand-limits`
- `-noskin`
- `-no-skin`
- `-onion`
- `-onion-alpha`
- `-onion-rotation`
- `-radio`
- `-ranchu`
- `-raw-keys`
- `-scale`
- `-shared-net-id`
- `-shell-serial`
- `-skin`
- `-skindir`
- `-trace`
- `-useaudio`

Get help about command-line options

This section describes how to get help about the command-line options. It provides more in-depth information about the commonly used emulator command-line options that are available when you start the emulator.

List all emulator options

To print a list of all emulator options, including a short description, enter the following command:

```
$ emulator -help
```

Get detailed help for a specific option

To print help for a specific startup option, enter the following command:

```
$ emulator -help-option 
```

For example:

```
$ emulator -help-netspeed
```

This help is more detailed than the description provided by the `-help` option.

Get detailed help for all options

To get detailed help for all emulator options, enter the following command:

```
$ emulator -help-all
```

List emulator environment variables

To get a list of emulator environment variables, enter the following command:

```
$ emulator -help-environment
```

You can set environment variables in the terminal window before launching a virtual device, or you can set it through your user settings in the operating system. For example,

set it in your `.bashrc` file on Linux.

List debug tags

To print a list of tags for the `-debug` options, enter the following command:

```
$ emulator -help-debug-tags
```

The `-debug` options let you enable or disable debug messages from specific emulator components, as specified by the tags.

Content and code samples on this page are subject to the licenses described in the [Content License \(/license\)](#). Java and OpenJDK are trademarks or registered trademarks of Oracle and/or its affiliates.

Last updated 2024-06-13 UTC.